

Data Analysis with Python 2024–2025

Parte 5: Machine Learning (Regressão Linear)

Soluções dos Exercícios

Soluções independentes elaboradas para fins de estudo, com base nos enunciados originais

Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**. As soluções abaixo não são o gabarito oficial do curso.

Conteúdo

1 Soluções - Regressão Linear

A seguir estão as soluções propostas para os exercícios sobre Machine Learning e Regressão Linear.

Exercício 5.10 – Regressão Linear

Solução proposta

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

def fit_line(x, y):
    model = LinearRegression(fit_intercept=True)
    # Adicionando um novo eixo para converter array 1D para 2D
    # exigido pelo sklearn
    model.fit(x[:, np.newaxis], y)
    return model.coef_[0], model.intercept_

def main():
    x = np.array([1, 2, 3, 4, 5])
    y = np.array([2.5, 3.1, 4.8, 5.2, 6.7])

    slope, intercept = fit_line(x, y)
    print(f"Slope: {slope}")
    print(f"Intercept: {intercept}")

    plt.plot(x, y, 'o', label='Pontos originais')
    plt.plot(x, slope * x + intercept, color='red', label='Linha
    ajustada')
    plt.legend()
    plt.show()

if __name__ == "__main__":
    main()
```

Esta solução instancia um objeto `LinearRegression` para calcular a linha que melhor se ajusta aos dados. Em seguida, extraímos o primeiro valor dos coeficientes gerados e o intercepto para retorná-los como tupla.

Exercício 5.11 – Dados Misteriosos (Mystery Data)

Solução proposta

```
import pandas as pd
from sklearn.linear_model import LinearRegression

def mystery_data():
    df = pd.read_csv("mystery_data.tsv", sep="\t")
    # As 5 primeiras colunas são os atributos (features) e a última
    # é a resposta (Y)
    X = df.iloc[:, 0:5]
    y = df.iloc[:, 5]

    model = LinearRegression(fit_intercept=False)
    model.fit(X, y)
    return model.coef_

def main():
    coefficients = mystery_data()
    for i, coef in enumerate(coefficients, 1):
        print(f"Coefficient of X{i} is {coef}")

if __name__ == "__main__":
    main()
```

O parâmetro `sep="\t"` é utilizado na leitura do arquivo TSV. Dividimos o Data-Frame nas variáveis explicativas (X) e na variável dependente (y). Foi configurado `fit_intercept=False` de acordo com as instruções.

Exercício 5.12 – Coeficiente de Determinação (R2)

Solução proposta

```
import pandas as pd
from sklearn.linear_model import LinearRegression

def coefficient_of_determination():
    df = pd.read_csv("mystery_data.tsv", sep="\t")
    X = df.iloc[:, 0:5]
    y = df.iloc[:, 5]

    model = LinearRegression(fit_intercept=True)

    # Avaliando todas as features juntas
    model.fit(X, y)
    scores = [model.score(X, y)]

    # Avaliando as features isoladamente
    for i in range(5):
        X_single = X.iloc[:, i].values.reshape(-1, 1)
        model.fit(X_single, y)
        scores.append(model.score(X_single, y))

    return scores

def main():
    scores = coefficient_of_determination()
    print(f"R2-score with feature(s) X: {scores[0]}")
    for i, score in enumerate(scores[1:], 1):
        print(f"R2-score with feature(s) X{i}: {score}")

if __name__ == "__main__":
    main()
```

Para treinar as features de modo avulso, é preciso isolá-las iterando sob as colunas do conjunto X e alterando o formato utilizando `reshape(-1, 1)`, um requisito para arrays únicos no Scikit-learn.

Exercício 5.13 – O Clima de Ciclismo Continua

Solução proposta

```
import pandas as pd
from sklearn.linear_model import LinearRegression

def cycling_weather_continues(station):
    # Lendo os dados
    weather = pd.read_csv("src/weather_data.csv")
    cycling = pd.read_csv("src/cycling_data.csv")

    # Restringindo os dados ao ano de 2017
    cycling = cycling[cycling['Year'] == 2017]

    # Agrupando e somando os contadores diários de ciclismo
    cycling_daily = cycling.groupby(['Year', 'Month', 'Day']).sum()
    .reset_index()

    # Mesclando os dados por Ano, Mes e Dia
    merged = pd.merge(weather, cycling_daily, on=['Year', 'Month',
        'Day'], how='inner')

    # Preenchendo os espaços vazios usando "forward fill"
    merged = merged.fillna(method='ffill')

    # Separando variáveis dependentes e independentes
    X = merged[['Precipitation amount (mm)', 'Snow depth (cm)', '
        Air temperature (degC)']]
    y = merged[station]

    # Treinando o modelo
    model = LinearRegression(fit_intercept=True)
    model.fit(X, y)

    return model.coef_, model.score(X, y)

def main():
    station = 'Baana'
    coefs, score = cycling_weather_continues(station)

    print(f"Measuring station: {station}")
    print(f"Regression coefficient for variable 'precipitation': {
        coefs[0]:.1f}")
    print(f"Regression coefficient for variable 'snow depth': {
        coefs[1]:.1f}")
    print(f"Regression coefficient for variable 'temperature': {
        coefs[2]:.1f}")
    print(f"Score: {score:.2f}")

if __name__ == "__main__":
    main()
```

O método `groupby` junto do `sum()` nos permite somar todas as medições de um mesmo dia para criar uma agregação correta. Após isso, usamos `pd.merge` e `fillna(method='ffill')` para alinhar o ciclo com os dados de clima correspondentes e limpar as lacunas.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.